AI Investment Committee

AAI-520 Final Team Project Proposal

Team: Group 7
Members: Guna Pasupathy, Mina Habib, and Michael Valderrama
Current stage: Proposal
Repository: AAI-520 Final Project

This proposal describes our starting idea for the final project. We expect the design to change as we learn transformers, LLMs, prompt engineering, and agentic AI in later modules.

1. Project idea and current course context

Our idea is to build an AI-assisted investment research system. A user will enter a stock ticker, and the system will collect a small amount of public market and news information. It will organize the information into positive and negative points and produce a short research memo. This is a course demonstration and not financial advice.

We are currently in Module 2. So far, the course has covered NLP fundamentals, text preprocessing, and news classification, and we are now learning named entity recognition (NER) and part-of-speech tagging. The final assignment also asks for concepts such as planning, routing, prompt chaining, evaluation, and memory. Those topics are scheduled for later in the course. At this stage, we understand them as project requirements, but we have not finalized how we will implement them.

Our main question is whether dividing the research into smaller steps can produce an investment summary that is easier to follow and check than a single generated response.

2. Project goals

Committed goals for Module 7

These goals come from the final-project instructions. The technical approach may change as we cover the later modules.

1. Accept a stock ticker and create a basic research plan.
2. Use public data tools to collect price, company, and news information.
3. Process news through the required steps: ingest, preprocess, classify, extract, and summarize.

4. Route information to an appropriate research role or specialist.
5. Create both positive and negative views of the company.
6. Review the first draft and revise weak or unsupported statements.
7. Save a short note that can be used during a later run.
8. Demonstrate the completed workflow and its limitations in the final notebook.

Stretch goals

- Add SEC filings or economic indicators as extra sources. **(stretch)**
- Compare more than one model or classification approach. **(stretch)**
- Build a more advanced method for checking generated claims. **(stretch)**
- Add a small user interface for entering a ticker. **(stretch)**

We will attempt stretch goals only after the required workflow is working.

3. Proposed workflow

The pseudocode below is our current high-level view of the project. It is not intended to be the final implementation.

```
START with a stock ticker

Read a note from an earlier run, if one exists
Create a short research plan
Collect public market, company, and news data

For each news article:
    clean the text
    classify the article
    identify names, companies, dates, and financial values
    create a short summary
    send it to the most relevant analysis step

Combine the collected findings
Create a positive view and a negative view
Write a first draft of the investment memo
Record the draft's confidence and any open gaps

Repeat until the draft is accepted or the revision limit is
reached:
    review each statement against its source
    if the statement is supported, accept it
    otherwise:
        send it back with a reason
        revise the statement

Save a short note for the next run
```

```
RETURN the final memo and the sources used
```

This flow keeps self-reflection and the review loop visible as separate steps. It includes the main behaviors required by the assignment, but we will decide the exact number of agents, prompts, and review passes after those topics are introduced in the course.

4. Current design thinking

What we can begin with now

Using the concepts from Modules 1 and 2, we can begin collecting sample news, cleaning the text, removing duplicates, and extracting useful entities such as company names, dates, people, and dollar amounts. We can also set up the repository and agree on a simple format for passing data between our parts of the project.

What we expect to add later

Later modules cover transformers, LLMs, RAG, prompt engineering, Hugging Face, and agentic AI. We expect those topics to help us choose a classification method, generate summaries, design the research roles, and build the review-and-revision step.

Decisions we are leaving open

We have not selected the final language model, orchestration framework, prompts, evaluation rules, or number of specialist agents. We also do not yet know whether every step needs an LLM. We will use simpler Python or NLP methods where they are enough and revise the design during the project check-ins.

After the required workflow is complete, we may add SEC or economic data **(stretch)**, compare another model or classifier **(stretch)**, test a more advanced claim-checking method **(stretch)**, or create a simple ticker-entry interface **(stretch)**.

Related work

Two projects we plan to use as background are TradingAgents, a multi-agent financial analysis framework described by Xiao et al. (2024), and the multi-agent debate approach studied by Du et al. (2023). We will use them as references rather than trying to reproduce their full systems.

5. Planned data, tools, and storage

Need	Current idea	Notes
Market and company data	`yfinance`	Recommended in the project instructions and does not require a paid account
Company news	Yahoo Finance news or another public source	We will keep a small saved sample in case the live source is unavailable
Text processing and NER	Python and spaCy	Fits the material being covered in Modules 1 and 2
Later language-model work	To be decided	We will choose after the related course modules
Additional SEC or economic data **(stretch)**	SEC EDGAR or FRED	Consider only after the required data workflow works
Alternative model comparison **(stretch)**	To be decided	Compare only if time and computing resources allow
Local storage	SQLite	Stores run notes, evidence references, and basic logs in one local file
Replay data	Sanitized JSON or CSV fixtures	Versioned sample responses can rebuild SQLite without an API key

Why we plan to use SQLite

We first considered hosted databases such as Supabase or Neon. They would make remote access easier, but they also require an account, credentials, network access, and database permissions. That could create unnecessary setup for the professor or anyone else trying to run the notebook.

SQLite is enough for the small amount of data in this project. It runs locally, does not require a username or password, and is included with Python. The notebook can create the database file and its tables automatically. This should make the final submission easier to reproduce. We can reconsider a hosted database only if collaboration becomes difficult with the local approach.

Guna will own the SQLite schema and the code that creates the database. Other project components will use that module instead of creating or changing tables directly. Generated `.db`, `.db-shm`, and `.db-wal` files will not be committed. Small sample inputs and sanitized replay fixtures will be committed as JSON or CSV files, and the notebook will use them to build SQLite on its first run.

6. Division of work

Now that Mina has joined the team, we plan to use one primary owner for each area. This should reduce overlapping edits while still allowing each change to be reviewed through

a pull request. The split may change as later course topics are introduced.

Team member	Primary ownership	Near-term handoff
Guna	Shared record models, SQLite schema and storage module, sample market and earnings data, market and earnings tools, news fetching, planner, and routing	Complete the shared models and storage layer first; then connect planner and router stubs to the tools
Mina	News chain: ingest, preprocess, classify, extract, and summarize; connection from processed news to shared evidence; evaluation harness and charts	Review the existing news chain, add the classification gate, and connect it to the shared models; then build the with-and-without-review measurement
Michael	Repository and CI, Bull and Bear analysts, claim review and revision loop, self-reflection and memory note, and final notebook assembly	Build analyst and evaluator stubs that use the shared evidence and claim records; then assemble the walking skeleton

The shared record models and storage interface are the first handoff because the other components depend on their field names and database functions. Mina can develop the news chain with saved articles while those interfaces are being completed, and Michael can prepare analyst stubs against the agreed record shapes. All three team members will review tests, evaluation results, the final notebook, and the PDF so that everyone can explain the complete project. Major design decisions will be made together during regular check-ins.

7. Learning and development plan

The module column connects the plan to the course, while the dated milestones describe what we want running by each Tuesday meeting.

Tuesday check-in	Course context	Milestone
Sep 15	Module 2	Repository available with CI and branch rules, proposal agreed, and owners named
Sep 22	Module 3	Data layer complete, sample data committed, SQLite builds on first run, and the news chain runs on saved articles
Sep 29	Module 4	Walking skeleton runs end to end with unfinished steps stubbed; Status Update Form submitted Sep 28
Oct 6	Module 5	Working planner, Bull and Bear roles, and one model connected
Oct 13	Module 6	Review loop and memory note working; with-and-without-review measurement completed

Tuesday check-in	Course context	Milestone
Oct 20	After Module 7	Final submitted Oct 19; peer evaluation and project retrospective

This schedule gives us an end-to-end structure early. As we reach Modules 5 and 6, we can replace stubbed steps with the approaches taught in class instead of waiting until the last week to integrate everything. Stretch goals will be considered only after the committed workflow is complete.

8. Evaluation, risks, and expected deliverables

For the final project, we plan to show the output from each required stage and test the system on a small set of stock tickers. The drafting step will record its own confidence and open gaps to demonstrate self-reflection. We will then run the same tickers with and without the review loop and count supported and unsupported statements. This will show whether the evaluator-optimizer loop improves the draft. We will also check whether plans change across companies, whether information is routed to different roles, and whether a saved note affects a later run.

Each model call will be logged in SQLite with its prompt, response, token count, and latency. For the final no-key replay, selected responses will be sanitized and stored as versioned JSON or CSV fixtures that build a fresh local database. The `.db` file itself will not be committed. Any extra-source, model-comparison, interface, or advanced-checking work will be reported separately as **(stretch)** and will not be required for the core evaluation.

Our main risks are unreliable public APIs, limited time, changing model outputs, and a design that becomes too complicated. We will keep saved sample data for repeatable testing, finish required features before stretch goals, and prefer simple components when they meet the requirement.

The expected final deliverables are a GitHub repository with a README, an executable code notebook, any small supporting Python files or sample data, evaluation results, limitations, and a PDF or HTML export of the final notebook.

AI-assistance disclosure

ChatGPT was used to provide a high-level overview of Modules 3 through 7, which helped us align the project plan with the topics covered later in the course. Claude was used by Michael to review the proposal and suggest improvements to the milestones, evaluation plan, and AI disclosure. The team reviewed the suggestions and decided which changes to include.